SHORT WEEK 1 PLAN

Popular Frontend and Backend Technologies

Simple English Study Data for a Software Developer

August 10-16, 2026 | 60 minutes per day | 7 hours total

Main target: Learn what the technologies do. Do not try to master every framework in one week.

Main stack to focus on after this week

- **Frontend** - HTML, CSS, TypeScript, React, Next.js
- **Backend** - Node.js, Express first; NestJS later
- **Database** - MySQL now; learn PostgreSQL concepts too
- **Tools** - Git, Docker, automated testing, CI/CD

How to study

[ ] Read the page once.
[ ] Type one small example.
[ ] Say the interview answer aloud three times.
[ ] Write three new English words.

Important: Popularity changes. This guide uses current survey signals and official documentation, but your project needs and local job market are also important.

First-Week Schedule

One short topic each day

Day	Topic	Concepts
Mon	How the web works	Frontend, backend, browser, server, HTTP
Tue	Frontend foundations	HTML, CSS, JavaScript, TypeScript, npm, Vite
Wed	Frontend frameworks	React, Next.js, Vue, Angular
Thu	Backend technologies	Node.js, Express, NestJS, Python, Java, C#
Fri	APIs and databases	REST, JSON, CRUD, SQL, MySQL, PostgreSQL, MongoDB, auth
Sat	Full-stack architecture	Connect UI, API, database, Git, Docker, cloud
Sun	Review and choose	Quiz, interview English, next stack

Daily 60-minute method

- **10 min** - Read and underline new words.
- **15 min** - Use one official tutorial.
- **20 min** - Type a tiny example or draw a diagram.
- **10 min** - Practice one interview answer aloud.
- **5 min** - Write what you learned.

Week 1 success

[ ] Explain frontend versus backend.
[ ] Explain TypeScript, React, Next.js, Node.js, REST, and SQL.
[ ] Draw one full-stack request flow.
[ ] Choose one stack for deeper study.

Day 1: How the Web Works

Monday, August 10, 2026

Goal: Understand how a click becomes data on the screen.

Study plan

10 min concepts | 15 min DevTools | 20 min diagram | 10 min interview | 5 min notes

Learn these concepts

- **Frontend** - The user interface in the browser. Example: A workspace page.
- **Backend** - Server code for rules, security, and data. Example: A workspace API.
- **Browser** - Displays HTML/CSS and runs JavaScript. Example: Chrome.
- **Server** - Receives requests and sends responses. Example: Node.js server.
- **HTTP** - Rules for web communication. Example: GET /api/workspaces.
- **Request/response** - The client asks; the server answers. Example: JSON plus a status code.

Practice

1. Open DevTools Network on a website and find one request.
2. Draw: Browser -> Frontend -> API -> Database -> API -> Screen.

Interview practice

Q: What is the difference between frontend and backend?
Simple answer: Frontend is the interface users see. Backend handles business rules, security, and database work.

Finish today

[ ] I can draw the request flow.
[ ] I can explain request and response.

Official resources: MDN Web | HTTP

Day 2: Frontend Foundations

Tuesday, August 11, 2026

Goal: Know the job of each basic frontend technology.

Study plan

10 min HTML/CSS/JS | 15 min TypeScript | 20 min small page | 10 min tools | 5 min notes

Learn these concepts

- **HTML** - Defines page structure and meaning. Example: Button, form, heading.
- **CSS** - Controls layout and style. Example: Responsive cards and dark mode.
- **JavaScript** - Adds behavior and browser logic. Example: Open a modal or fetch data.
- **TypeScript** - JavaScript with type checking. Example: Workspace has id and name.
- **npm** - Installs and manages JavaScript packages. Example: npm install react.
- **Vite** - A fast development server and build tool. Example: Starts a React project.

Practice

1. Create a page with a heading, input, button, CSS card, and one JavaScript action.
2. Write a TypeScript interface for Workspace.

Interview practice

Q: Why do developers use TypeScript?
Simple answer: It catches many mistakes before runtime, improves editor help, and documents data shapes.

Finish today

[ ] My small page works.
[ ] I can explain JavaScript versus TypeScript.

Official resources: MDN HTML/CSS/JS | TypeScript | Vite

Day 3: Frontend Frameworks

Wednesday, August 12, 2026

Goal: Understand React and Next.js, and recognize Vue and Angular.

Study plan

10 min components | 15 min props/state | 15 min comparison | 15 min example | 5 min notes

Learn these concepts

- **Component** - A reusable UI piece. Example: WorkspaceCard.
- **Props** - Data passed into a component. Example: name and description.
- **State** - Data that changes during interaction. Example: modalOpen.
- **React** - A component-based UI library. Example: Build dashboards and forms.
- **Next.js** - A React framework with routing and server features. Example: Pages, layouts, data fetching.
- **Vue/Angular** - Other important frontend frameworks. Example: Vue is approachable; Angular is structured.

Practice

1. Write a TechnologyCard component with name and purpose props.
2. Write one sentence comparing React and Next.js.

Interview practice

Q: What is the difference between React and Next.js?
Simple answer: React builds user interfaces. Next.js adds routing, server rendering, data fetching patterns, and production tooling.

Finish today

[ ] I understand component, props, and state.
[ ] I can compare React, Next.js, Vue, and Angular.

Official resources: React | Next.js | Vue | Angular

Day 4: Backend Technologies

Thursday, August 13, 2026

Goal: Know the major backend choices and their main purpose.

Study plan

10 min backend jobs | 20 min Node/Express | 15 min alternatives | 10 min endpoint | 5 min notes

Learn these concepts

- **Node.js** - Runs JavaScript or TypeScript on the server. Example: REST API.
- **Express** - A small, flexible Node.js web framework. Example: Routes and middleware.
- **NestJS** - A structured TypeScript backend framework. Example: Modules and controllers.
- **FastAPI/Django** - Popular Python choices. Example: APIs or full web applications.
- **Spring Boot** - A major Java backend framework. Example: Enterprise services.
- **ASP.NET Core/Laravel** - Major C# and PHP frameworks. Example: Business web systems.

Practice

1. Write pseudocode for GET /api/workspaces.
2. List backend jobs: validation, authorization, database, logging, errors.

Interview practice

Q: Why use Node.js for a backend?
Simple answer: It lets a team use JavaScript or TypeScript across the stack and has a large ecosystem for APIs and real-time systems.

Finish today

[ ] I can name four backend framework choices.
[ ] I understand route and middleware.

Official resources: Node.js | Express | NestJS | FastAPI

Day 5: APIs, Databases, and Authentication

Friday, August 14, 2026

Goal: Understand communication, storage, and permission.

Study plan

10 min REST/JSON | 15 min CRUD | 15 min databases | 15 min auth | 5 min notes

Learn these concepts

- **API** - A defined way for software to communicate. Example: /api/workspaces.
- **REST/CRUD** - Resource operations using HTTP. Example: POST, GET, PATCH, DELETE.
- **JSON** - A common API data format. Example: {"id":42,"name":"Study"}.
- **SQL** - Queries relational tables. Example: Users and workspaces.
- **PostgreSQL/MySQL** - Popular relational databases. Example: Reliable app data.
- **Authentication/authorization** - Who are you, and what may you do? Example: Owner may delete workspace.

Practice

1. Design list, create, update, and delete workspace endpoints.
2. Draw users and workspaces tables with owner_id.

Interview practice

Q: Authentication versus authorization?
Simple answer: Authentication checks identity. Authorization checks permission.

Finish today

[ ] I can map CRUD to HTTP methods.
[ ] I can explain SQL and permission basics.

Official resources: REST | PostgreSQL | MySQL | OWASP Auth

Day 6: Full-Stack Architecture

Saturday, August 15, 2026

Goal: Connect frontend, backend, database, and delivery tools.

Study plan

15 min flow | 20 min design | 20 min Git/Docker/cloud | 20 min mini system | 15 min interview

Learn these concepts

- **Architecture** - The high-level structure of the system. Example: UI, API, database.
- **Git/GitHub** - Track and review code changes. Example: Branches and pull requests.
- **Docker** - Packages code and dependencies. Example: Repeatable containers.
- **CI/CD** - Automatically checks, builds, tests, and deploys. Example: GitHub Actions.
- **Cloud** - Hosts apps, databases, files, and monitoring. Example: AWS, Azure, or Google Cloud.
- **Environment variable** - Configuration outside source code. Example: DATABASE_URL.

Practice

1. Draw Next.js -> Node API -> MySQL and add authentication.
2. Explain the Edit Workspace request from click to database and back.

Interview practice

Q: Describe your project architecture.
Simple answer: The React or Next.js frontend calls a Node.js REST API. The API validates input, checks permission, and reads or updates MySQL. Git tracks changes, tests protect behavior, and Docker makes environments repeatable.

Finish today

[ ] I created one architecture diagram.
[ ] I can explain one full request flow.

Official resources: Git | Docker | GitHub Actions | AWS

Day 7: Review and Choose Your Stack

Sunday, August 16, 2026

Goal: Check your knowledge and select the next learning focus.

Study plan

15 min quiz | 10 min comparison | 10 min interview English | 10 min next plan

Learn these concepts

- **Best current fit** - TypeScript + React + Node.js + MySQL. Example: Matches your Trello project.
- **Next.js** - Learn after React basics. Example: Routing and server features.
- **Express first** - Good for learning API basics. Example: Routes and middleware.
- **NestJS later** - Useful when you want stronger structure. Example: Modules and dependency injection.
- **Testing** - Jest, Testing Library, and Playwright. Example: Unit, component, and E2E tests.
- **Deployment** - Docker and GitHub Actions later. Example: Repeatable releases.

Practice

1. Answer: frontend/backend, TypeScript, React/Next.js, Node.js, REST, SQL, authentication.
2. Say your 30-second developer introduction three times.

Interview practice

Q: What stack are you focusing on now?
Simple answer: I am focusing on TypeScript, React, Node.js, and MySQL because they match my full-stack project. I will add Next.js, testing, Docker, and CI/CD step by step.

Finish today

[ ] I answered at least five questions aloud.
[ ] I chose one stack for the next three weeks.

Official resources: React | Node.js | TypeScript

Quick Technology Comparison

Review page - do not memorize every detail

Frontend

Technology	What it is	Main idea
React	UI library	Flexible, large ecosystem
Next.js	React framework	Routing, server rendering, production features
Vue	Framework	Approachable and reactive
Angular	Full TypeScript framework	Strong structure for large applications

Backend

Framework	Language	Main idea
Express	Node.js	Minimal and flexible
NestJS	TypeScript	Structured and scalable
FastAPI	Python	Modern APIs and automatic docs
Django	Python	Many built-in web features
Spring Boot	Java	Enterprise backend ecosystem
ASP.NET Core	C#	Strong .NET web platform
Laravel	PHP	Productive business web development

Database

- **PostgreSQL** - Powerful general-purpose relational database.
- **MySQL** - Widely used relational database and your current choice.
- **MongoDB** - Document database for JSON-like data.
- **Redis** - Fast in-memory storage for cache, sessions, and queues.

One simple decision rule

Choose: Learn one stack deeply. Recognize other stacks, but do not switch every week.

Glossary, Sources, and Next Step

Keep this page for daily review

12 important words

Word	Simple meaning
Architecture	system structure
Component	reusable UI piece
Endpoint	API address and operation
Framework	tools and conventions for building software
Library	reusable code your app calls
Middleware	code in the request pipeline
Render	create visible UI
Route	URL connected to a page or handler
Schema	defined data structure
State	changing application data
Validation	check input rules
Deployment	release software to an environment

Next three weeks

- **Week 2** - JavaScript and TypeScript foundations.
- **Week 3** - React components, props, state, forms, and API calls.
- **Week 4** - Node.js, Express, REST APIs, validation, and MySQL integration.

Popularity note

This is a practical overview, not an official ranking. The technology choices are informed by the 2025 Stack Overflow Developer Survey, GitHub Octoverse 2025, and official documentation. GitHub reported that TypeScript became its most-used language in August 2025. Technology choice should also consider the project, team, jobs, and learning goals.

Main sources

- Stack Overflow Survey 2025
- GitHub Octoverse 2025
- MDN Web Docs
- React Docs
- Node.js Learn
- TypeScript Handbook

Study promise: Use short English sentences. Repeat the same technical explanation many times. Improve one small step every day.